Önálló laboratórium beszámoló

Távközlési és Mesterséges Intelligencia Tanszék

Név:	**Orbán Levente**
	orban.levente.laszlo@gmail.com
Neptun-kód:	**CZ6NDB**
Szakirány:	**Szoftverfejlesztés**
Konzulens:	**Németh Gábor**
	nemethgab@tmit.bme.hu

Téma címe: Interaktív SKI-kombinátor kalkulus fordító és vizualizáló

Feladat:
A hallgató feladata egy webes eszköz (Combinator Explorer) elkészítése, amely SKI-kifejezések esetén megmutatja a redukciós lépéseket, vizualizálja a redexeket, és felrajzolja a kiértékelési fát. A redukció során többféle kiértékelési stratégiát támogat, valamint az SKI-kifejezéseket lefordítja λ-kalkulusba. Az eszköz hidat képez a formális elmélet és a gyakorlati megvalósítás között: oktatási segédeszköz, amely mélyíti a funkcionális programozás, illetve a fordítóprogramok működésének megértését.

Tanév: 2025/26. tanév, II. félév

1. A laboratóriumi munka környezetének ismertetése, a munka előzményei és kiindulási állapota

1.1. Bevezető

Az elmúlt három évtizedben egyre nagyobb jelentőségre tesz szert a funkcionális programozás az iteratív és objektumorientált paradigmákkal szemben. Az figyelhető meg, hogy azok a programozási nyelvek, amelyek tradicionálisan objektumorientált szemléletűek voltak (például C++, Java), azokat napjainkra jelentős számú funkcionális elemmel bővítették (anonim függvények, lista komprehenzió, első osztályú függvények, etc.). Továbbá azok a nyelvek, amelyek az elmúlt évtizedben tettek szert népszerűségre (például Rust) már a kezdetektől fogva egy multiparadigmás nézettel lettek tervezve. Ezeket az új eszközöket viszont – mind a már piacon lévő mérnökök továbbképzése, mind a felsőoktatásban résztvevők oktatása szempontjából – pedagógiailag még a mai napig nehéz kielégítően oktatni. A funkcionális paradigma – illetve annak a matematikai alapjai – idegenek lehetnek a mérnöki beállítottságú közönség számára.

A jelen írásos beszámolóban bemutatott szoftver ezt a pedagógiai hiányt próbálja pótolni: egy olyan vizuális, interaktív, webes eszközt ad a pedagógusok kezébe, amellyel a funkcionális programozást egészen a matematikai alapokig visszamenően lehet hatékonyabban oktatni, hozzá pontosabb intuíciót építeni.

A választott megközelítés (vizuális, webes program) nem egyedi. Számtalan más olyan témakörhöz, amelyek tradicionálisan pedagógiai kihívásokat jelentettek (generált gépi kód, vagy reguláris kifejezések elemzése) készült hasonló alapötletekkel rendelkező szoftver (mint például a Compiler Explorer, vagy Regex101).

A jelen beszámolóban bemutatott megközelítés a modern funkcionális nyelvek matematikai alapjait kívánja megcélozni (többek között redukciós láncok és gráfok, kiértékelési sorrendek, λ- és SKI-kalkulus). Ezen alaptudás ismeretében a legtöbb modern funkcionális nyelv (ML-család, Lisp-család) értelmezőjének a működése – legalábbis magas szinten – könnyebben elsajátítható.

1.2. Elméleti összefoglaló

1.2.1. λ-kalkulus

A modern funkcionális programozási nyelvek alapjaikban a (típus nélküli) λ-kalkulusra építenek[1], amely egy általános célú, Turing-teljes programozási nyelv. Az egyik legegyszerűbb Turing-teljes nyelv, hiszen csak három konstrukcióra épít (lambda-kifejezések, lambda-absztrakciók, és applikációk). A λ-kalkulus nyelvtanának formális definíciója az 1. ábrán látható.

$$
\begin{array}{lcl}
\langle \lambda\text{-}\textit{kifejezés} \rangle & ::= & \langle \textit{változó} \rangle \\
 & | & \langle \lambda\text{-}\textit{absztrakció} \rangle \\
 & | & \langle \textit{applikáció} \rangle \\
 & & \\
\langle \lambda\text{-}\textit{absztrakció} \rangle & ::= & \text{‘}\lambda\text{’}\ \langle \textit{változó} \rangle\ \text{‘.’}\ \langle \lambda\text{-}\textit{kifejezés} \rangle \\
 & & \\
\langle \textit{applikáció} \rangle & ::= & \langle \lambda\text{-}\textit{kifejezés} \rangle\ \langle \lambda\text{-}\textit{kifejezés} \rangle
\end{array}
$$

1. ábra. A λ-kalkulus szintaktikája

A λ-absztrakció segítségével tudunk függvényeket definiálni, amelyeknek paramétereket az applikáció konstrukciójával tudunk adni. Ha kicsit módosítjuk a nyelvtanunkat, hogy szám konstansokat, és egyéb kényelmes konstrukciókat megengedjen, akkor írhatjuk például:

$$
\begin{array}{l}
\lambda x.(5 * x) \\
(\lambda x.(5 * x))(3) \\
15
\end{array}
$$

[1] Tekinthetőek a λ-kalkulus kibővített, változatának (legyen szó típusokról, vagy let-kötésekről, etc.). Példa erre az ML-család (Standard ML, Caml, Haskell), amelyek közelítőleg felfoghatóak úgy, mint a λ-kalkulus polimorf Hindley-Milner típusrendszerrel való kiegészítése.

Az első sorban definiálunk egy függvény, amely ötszörözi a bemenetét. A második sorban meghívjuk ezt a függvényt a 3 bemenettel. Az utolsó sorban az applikáció eredménye látható. Fontos megjegyezni, hogy az applikáció bal-asszociatív.

1.2.2. A SKI-kalkulus

A kombinátorok absztrakt matematikai konstrukciók, viszont megértésük és történelmi hátterük kritikus fontosságú a modern számítástudomány, különösen a funkcionális programozás, vagy a kiszámíthatóság modelljeinek vizsgálatakor.

Általánosságban problémák formális, szimbolikus ábrázolását már az ókorban megkísérelték. Így született Arisztotelész logikája és Euklidész geometriája. A számítástudomány szempontjából viszont a 19. században a Boole-algebra megjelenése jelentette a legnagyobb áttörést. A 20. század fordulóján a matematikusok arra tettek kísérletet, hogy a teljes matematikát egy szigorú, tiszta logikai alapra helyezzék.

Ahogy ezek a logikai keretrendszerek egyre nagyobbak lettek, megjelent az igény a bizonyos axiómák meghatározására. Ennek a törekvésnek a legismertebb példája a Sheffer-vonás felfedezése, amely bebizonyította, hogy a teljes ítéletkalkulus felépíthető egyetlen logikai kapu segítségével (NAND). Felmerült a kérdés: ha a logikai axiómák ilyen elemi szintre redukálhatóak, megtehető-e ugyanez általánosságban a függvényekkel és a változókkal is?

1920-ban Moses Schönfinkel pontosan erre tett kísérletet [4]. Felismerte, hogy ha a függvényeket kellőképpen általánosítjuk (egy függvény egy másik függvényt is kaphat paraméterként, és visszatérési értéke is lehet függvény), a változók feleslegessé válnak. Ezzel az elképzeléssel lefektette a kombinátorkalkulus alapjait.

A kombinátorkalkulus leghíresebb és leginkább letisztult formalizációja az SKI-kalkulus, amelyet Schönfinkel munkássága nyomán Haskell Curry dolgozott ki [1]. Ez pusztán három előre definiált kombinátor egymás utáni alkalmazásával is képes Turing-teljes számítások leírására.

A rendszer a következő három kombinátorból áll:

- Az identitás kombinátor (I), amely beavatkozás nélkül, egyszerűen visszaadja a kapott argumentumát.
- A konstans kombinátor (K), amely olyan függvényt állít elő, ami a második argumentumát mindig figyelmen kívül hagyja, és visszaadja az elsőt.
- A szubsztitúciós kombinátor (S), amely az argumentumok szétosztását és a függvényalkalmazás láncolatát végzi.

Bár a rendszert általában további kombinátorokkal is ki szokás egészíteni a kényelmesség és átláthatóság érdekében, ezek mind levezethetőek a fenti három kombinátorból.

Ha a λ-kalkulus, és a SKI-kalkulus és Turing-teljesek, akkor bizonyára kell lennie valamilyen ekvivalenciának közöttük. Alább megadom a SKI-kalkulus három kombinátorának a λ-kalkulus szerinti definícióját:

$$\begin{aligned} \mathsf{I} &= \lambda x.x \\ \mathsf{K} &= \lambda x.(\lambda y.x) \\ \mathsf{S} &= \lambda x.(\lambda y.(\lambda z.xz(yz))) \end{aligned}$$

Ezeknek a szabályoknak az alkalmazásával tetszőleges SKI-kalkulus kifejezést át lehet alakítani λ-kalkulus kifejezéssé, melyre a programban lehetőség is van.

1.3. A munka állapota, készültségi foka a félév elején

A jelen félév előtti szünetben már elkezdtem feldolgozni a témához releváns irodalmat. A 2025/26/I. félévben egy olyan tárgyat hallgattam, amelyen jelentős szerepe volt a λ-kalkulusnak, kategória elméletnek. Ezek az előadások nagyban hozzájárultak az előzetes tudás megszerzéséhez. A konzulensem számos releváns könyvet ajánlott feldolgozásra, többek között [5, 6, 7].

2. Az elvégzett munka és az eredmények ismertetése

2.1. Az architektúra megtervezése

Az architektúra tervezésének folyamatában fordítóprogramokból merítettem inspirációt, különös tekintettel a moduláris felépítésre és az úgynevezett *Separation of Concerns* elvére.

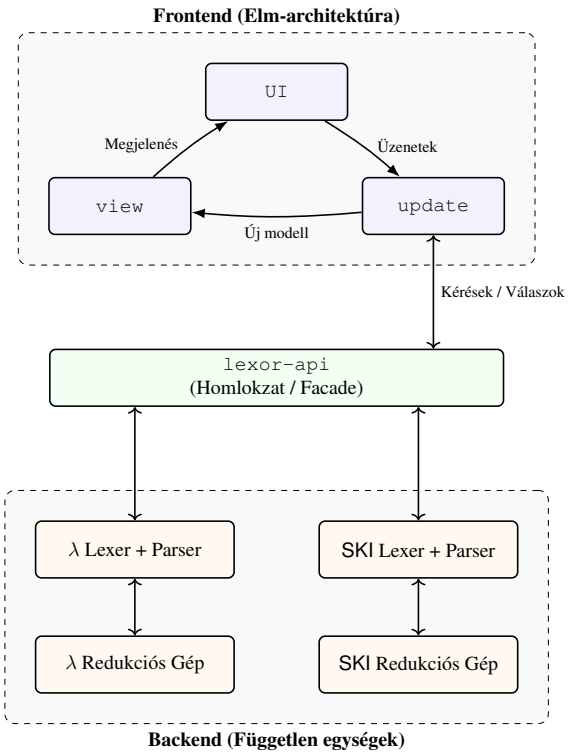
A nyelvtanok egyszerűsége miatt a lexert és a parsert egybevontam. A parserek megírásához a chumsky könyvtárat használtam, amely egy *parser combinator* alapú módszert használ, rekurzív leszállást igénybe véve. Két redukciós gépet implementáltam: egyet ami λ-kalkuluson, egyet ami SKI-kalkuluson operál. A két parser és a hozzájuk tartozó redukciós gépekre egy egységként tekintettem, ezek alkották a backend könyvtárat.

A frontend és a backend közötti tiszta kommunikáció megteremtése érdekében a Homlokzat (Facade) programtervezési mintát alkalmaztam. Ez az API réteg (`lexor-api`) elrejti a redukciós gépek komplexitását, és egy egységes interfészt biztosít a webes felület számára.

A frontend terén az úgynevezett *Elm-architektúrát* követtem, amely egy üzenetsor-alapú architektúra. Ez a megközelítés három részre bontja a frontendet:

- A kezdeti `model`-re, amely az applikáció állapotát írja le. Ez az állapot nem mutábilis, tehát ahogy a felhasználó interakcióba lép vele, mindig egy teljesen új állapot jön létre.
- A `view` függvényre, amely a modell megjelenését írja le. A felhasználói interakciókat itt alakítom át üzenetekké.
- Az `update` függvényre, amely az üzenetsort kiolvasva végrehajtja a felhasználó összes parancsát a modellen, egy új modellt létrehozva ezáltal.

Ahogy a felhasználó interakcióba lép a `UI`-al (User Interface), úgy új üzenetek kerülnek az üzenetsorba. Az üzenetsort minden frame végén az `update` függvény kiüríti, és a tartalmát végrehajtja. Ezáltal egy új modell jön létre, amit átad a `view` függvénynek, ami egy új `UI`-t hoz létre. Az architektúra egyszerűsített vizuális modellje a 2. ábrán látható.



2. ábra. Az alkalmazás rétegei és architektúrája

Mind a frontend, mind a backend WebAssembly modulokra kerül fordításra, tehát az egész program a felhasználó böngészőjében tud futni kliensoldalon.

2.2. A redukciós gépek implementálása

A redukciós gépek implementálásához a Graph Reduction Machine (G-machine) koncepcióját vizsgáltam meg. Optimalizálási célból a fa-alapú kiértékelés mellett az alkifejezések megosztása mellett döntöttem, amelyel exponenciálisan kevesebb lépésszámot vesznek igénybe az egyes redukciók [3]. Az elterjedtebb "sztringredukció" – azaz amikor a bemenetet szövegmanipulációs módszerekkel alakítjuk át a redukciós szabályoknak megfelelően – nem skálázódik elég jól hosszabb kifejezések kiértékelésére, illetve az előbb említett alkifejezés-megosztás technikáját sem tudom érvényesíteni.

A követelmények elemzése és a belső reprezentáció kialakítása után implementáltam egy G-machine-t mind a λ-kalkulus, mind a SKI-kalkulus esetében. A G-machine-eknél gyakran használt, fa alapú AST sokszor memóriatöredezettséghez és lassú futáshoz vezethet. Ennek elkerülése érdekében a Flat-AST (laposított AST) reprezentációs technikát használtam, amelyet egy aréna memória-allokátorral (Arena allocator) párosítottam [2]. Egy rekurzív, fa alapú megközelítés lehet az 1. listingben bemutatott modell. Itt pointere-

```
enum NodeRec {
    S, K, I,
    App(Box<Self>, Box<Self>),
    Indirection(Box<Self>),
}
```

1. Listing. Egy rekurzív, mutató alapú gráf struktúra

ket tárolunk el a többi csúcsra, amely csökkentheti a memória lokalitását, illetve az adatstruktúra méretét is növelheti 64-bites rendszereken. Ehhez képest az aréna-allokátoros megközelítés – melyet a 2. listing mutat be – nem használ mutatókat. Minden csúcsra az indexe alapján lehet hivatkozni, mely alapján az arénából kinyerhető.

```
enum NodeFlat {
    S, K, I,
    App(NodeIdx, NodeIdx),
    Indirection(NodeIdx),
}
```

2. Listing. Egy aréna-allokátoros gráf struktúra

A G-machine a kombinátorok redukcióját egy állapotgép segítségével végzi. A kiértékelés során a veremben (spine) összegyűlt argumentumokat a gép a megfelelő kombinátor szabályai szerint helyettesíti be (például az S, K, I kombinátorok esetében). A redukciós szabályok alkalmazását végrehajtó Rust kódrészlet a 3.2. szakaszban, a csatlakozó dokumentumok között tekinthető meg.

Mivel a szoftver célja az oktatás megkönnyítése, több redukciós stratégiát is támogatnom kellett (például Normal order, Applicative order, lusta kiértékelés). Ezek megértéséhez elemezzük a 3. ábrát [6], amely az alábbi kombinátor AST-ét ábrázolja:

$$\mathsf{S(S(S)(S)(S(S)(K(K)(S))(S)))(S)(S)(S(K(S)(K))(K)(S))}$$

Az ábrán látható fában a szürke belső csomópontok az applikációkat (függvényalkalmazásokat) jelölik, a levelek pedig az egyes kombinátorokat. Látható, hogy egy ilyen komplex kifejezés esetén egyszerre több ponton is elvégezhető lenne a redukció. A támogatott stratégiák pontosan abban térnek el, hogy ezek közül melyiket választják:

- Normál sorrend (Normal order / Call-by-name): Mindig a legbaloldalibb, legkülső (leftmost-outermost) redexet választja ki. A gráfban ez a gyökértől balra haladva a legmagasabban fekvő kiemelt csomópont. Ez a stratégia garantálja, hogy ha egy kifejezésnek létezik normálformája (végeredménye), akkor azt el is éri a redukció végeztével. Ez a 3. ábrán a $\mathsf{S(S(S)(S)(S(S)(K(K)(S))(S)))(S)(S)}$ alkifejezésnek felel meg.

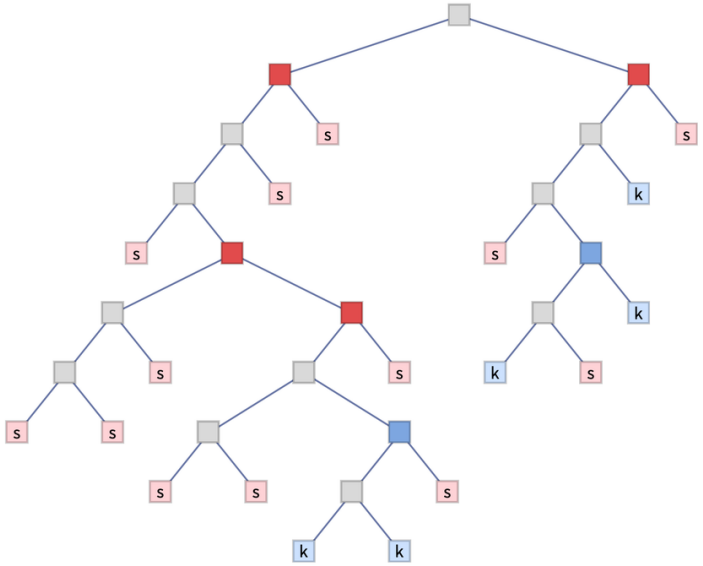
- Applikatív sorrend (Applicative order / Call-by-value): A legbaloldalibb, legbelső (leftmost-innermost) redexet redukálja először. A gyakorlatban ez azt jelenti, hogy egy függvény alkalmazása előtt a paramétereit (argumentumait) értékeli ki, a fában lentről felfelé haladva. Ez a 3. ábrán a $\mathsf{K}(\mathsf{K})(\mathsf{S})$ alkifejezésnek felel meg. A legtöbb modern imperatív és szigorú (strict) funkcionális nyelv (pl. C++, Rust, OCaml) ezt használja.
- Lusta kiértékelés (Lazy evaluation): Hasonló a normál sorrendhez, de a gráfredukció (sharing) segítségével megjegyzi a már kiértékelt alkifejezéseket a memóriában, így elkerüli a többszöri, redundáns kiértékelést (ezt a megközelítést alkalmazza a Haskell).

A stratégiák oktatása azért kiemelten fontos, mert vannak olyan SKI-kalkulus kifejezések, amelyeknek a redukciója bizonyos stratégiákat alkalmazva végtelen ciklusba kerül. Tekintsük például az alábbi kifejezést:

$$\mathsf{KI}((\mathsf{SII})(\mathsf{SII}))$$

Applikatív sorrend alkalmazásával, a gép először megpróbálja az argumentumokat $((\mathsf{SII})(\mathsf{SII}))$ teljesen kiértékelni, mielőtt a K kombinátort alkalmazná. Ez egy végtelen ciklust eredményez. Normál sorrend szerint viszont az algoritmus a legkülső redexet (a K applikációját) választja ki először, ezáltal a második argumentum (a divergáló kifejezés) eltűnik, és a kifejezés normál-formába kerül (I).

A kialakításban különösen figyeltem arra, hogy a redukciós logika moduláris maradjon, így az egyes stratégiák könnyen cserélhetők és összehasonlíthatók anélkül, hogy a motor alapvető működését módosítani kellene.



3. ábra. Egy kombinátor absztrakt szintaxis fája. A különböző kiértékelési módok kiválóan bemutathatóak rajta.

2.3. A tesztelő orákulum kialakítása

A szoftver helyességének verifikálására a hagyományos egységtesztek mellett a Property-based testing (tulajdonság-alapú tesztelés) módszertanát alkalmaztam. Ennek során a teszt-keretrendszer automatikusan generál valid, véletlenszerű λ- és SKI-kifejezéseket, majd megvizsgálja a rendszer invariáns tulajdonságait.

A redukciós gépek kimenetének validálásához egy tesztelő orákulumot alakítottam ki. Az orákulum egy már bizonyítottan helyesen működő, külső interpreter, amelyet egy elszigetelt Podman konténerben futtattam le a biztonság és a determinisztikus környezet garantálása érdekében. A tesztelési folyamat során a `lexor` és a Podman konténerben futó orákulum is kiértékelte ugyanazt a generált kifejezést, a teszt-keretrendszer pedig a két kimenet ekvivalenciáját vizsgálta Black-box (fekete doboz) tesztelés formájában.

2.4. A vizuális redukciós elemek létrehozása

Egy oktatási eszköz esetében a belső folyamatok láthatósága kritikus jelentőségű, ezért a backendet felkészítettem arra, hogy úgynevezett *callback* függvényeket tudjon fogadni. Ezek a függvények egy olyan nézetet kapnak a belső gépről, amelyen olyan metódusok vannak definiálva, amelyekkel a teljes AST-t be lehet járni, szemantikai információt fűzve az egyes csúcsokhoz. Így az egyes redukciós lépések könnyen naplózhatóak, az alkifejezések különböző szerepe pedig elkülöníthető.

Algoritmusokat dolgoztam ki a redukálható kifejezések (úgynevezett *redexek*) detektálására és strukturált adatként (JSON) történő továbbítására a frontend felé. Ezek az adatok teszik lehetővé, hogy a webes felület vizuálisan kiemelje a soron következő redukciós lépést.

Egy ilyen vizuális reprezentáció a 4. ábrán látható, amelyet [6, Wolfram] után vettem át. A különböző kombinátoroknak más-más színeik vannak, és meg vannak különböztetve a redex-ek az argumentumaiktól.

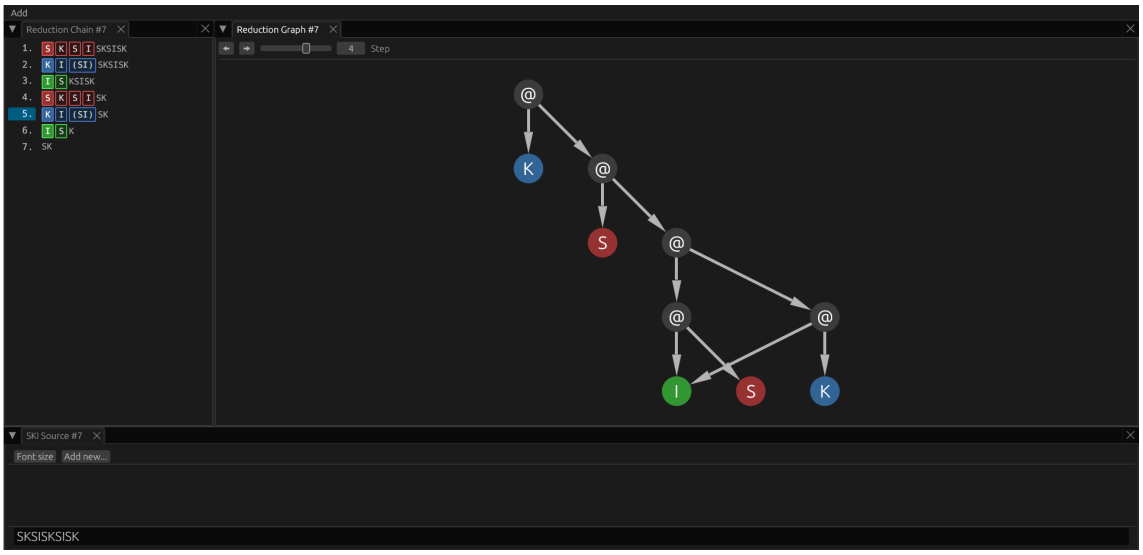
4. ábra. Egy redukciós lánc, ahol az egyes kifejezések szemantikailag vannak kiszínezve.

2.5. A frontend fejlesztése

A frontend egy teljes mértékben kliensoldalon futó alkalmazás, amelyet Rust nyelvből WebAssembly-re (WASM) fordítottam.

Az Elm-architektúrára épülő reaktív felületen a felhasználó interaktívan gépelhet be kifejezéseket, kiválaszthatja a kívánt kalkulust és a kiértékelési stratégiát. A gördülékeny felhasználói élmény érdekében a háttérfolyamatokat Service Worker-ekbe (`worker.js`) szerveztem. Így a komplex és esetlegesen hosszú ideig futó redukciós fák kiértékelése nem fagyasztja le a felhasználói felületet, és a szoftver akár offline módban (PWA) is képes teljeskörűen funkcionálni. A grafikus felületet az 5. ábrán mutatom be.

Az egui grafikus keretrendszert használtam a felület kialakítására, amely egy úgynevezett *immediate mode* grafikus felület. Ennek a megközelítésnek az a különlegessége, hogy a felhasználói felület minden egyes képkockánál a nulláról épül fel és rajzolódik ki. A gombok, vagy beviteli mezők állapota nincsen eltárolva a memóriában két egymást követő frame között. Ez azt is jelenti, hogy a felület kirajzolása, és a felhasználói interakciók kezelése egyetlen lépésben történik.



5. ábra. A grafikus felülete a programnak. A bal felső sarokban egy redukciós lánc látható. A képernyő alján a beviteli mező helyezkedik el, az aktuális bemenettel. A képernyő maradék részét a képernyőn kiválasztott kifejezés grafikonja tölti ki.

2.6. Összefoglalás

A félév során sikeresen megvalósítottam a munkatervben kitűzött célokat. Áttanulmányoztam a releváns szakirodalmat, ami alapján megterveztem az architektúrát. Kialakítottam a nyelvtanokat feldolgozó parsereket, implementáltam mind a λ-, mind a SKI-kalkulus G-machine alapú redukciós gépét, és mindezt egy WebAssembly alapú, interaktív frontenddel kötöttem össze. A Podman konténeres orákulumra épülő Property-based testing gondoskodik a motorok megbízhatóságáról. Az elkészült *Combinator Explorer* eszköz így sikeresen tölti be funkcióját: vizuális formát ad a funkcionális programozás matematikai alapjainak, hatékony és korszerű segédeszközt nyújtva a felsőoktatás számára.

3. Irodalom, és csatlakozó dokumentumok jegyzéke

3.1. A tanulmányozott irodalom jegyzéke

[1] H. B. Curry: Grundlagen der Kombinatorischen Logik. *American Journal of Mathematics*, 52. évf. (1930) 3. sz., 509–536. p. ISSN 0002-9327.

[2] Flattening ASTs (and Other Compiler Data Structures).
URL https://www.cs.cornell.edu/~asampson/blog/flattening.html.

[3] Lambda is not a four letter word - Implementing a Functional Language with Graph Reduction. URL https://thma.github.io/posts/2021-12-27-Implementing-a-functional-language-with-Graph-Reduction.html.

[4] M. Schönfinkel: Über die Bausteine der mathematischen Logik. *Mathematische Annalen*, 92. évf. (1924. szeptember) 3-4. sz., 305–316. p. ISSN 0025-5831, 1432-1807.

[5] Raymond M. Smullyan: *To Mock a Mockingbird: And Other Logic Puzzles Including an Amazing Adventure in Combinatory Logic*. 2000, Oxford University Press. ISBN 978-0-19-280142-5.

[6] Stephen Wolfram: Combinators: A Centennial View. *Stephen Wolfram Writings*, 2020. december.

[7] Stephen Wolfram: Combinators and the Story of Computation. *Stephen Wolfram Writings*, 2020. december.

3.2. A csatlakozó dokumentumok jegyzéke

Az alábbi kódrészlet a `lexor-reducer/src/core/engine.rs` fájlból származik. Azt a `reduce` függvényt és `match` blokkot mutatja be, amely az S, K, I, B és C kombinátorokhoz tartozó gráf-redukciós lépéseket (az új csomópontok allokálását és a mutatók átírását) végzi el az aréna-allokátoron belül.

```
        let result = match redex {
            NodeComb::I | NodeComb::K => Node::Indirection(args[0]),
            NodeComb::S => {
                let xz = self.arena.insert(Node::App(args[0], args[2]));
                let yz = self.arena.insert(Node::App(args[1], args[2]));
                Node::App(xz, yz)
            }
            NodeComb::B => {
                let yz = self.arena.insert(Node::App(args[1], args[2]));
                Node::App(args[0], yz)
            }
            NodeComb::C => {
                let xz = self.arena.insert(Node::App(args[0], args[2]));
                Node::App(xz, args[1])
            }
        };

        self.arena.replace(roots[0], result);
```

3. Listing. Kombinátorok redukciós szabályai a G-machine motorban